



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения
(ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: Проектирование и разработка серверных частей интернет-ресурсов

по профилю: Разработка и дизайн компьютерных игр и мультимедийных приложений

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: Серверная часть веб-приложения «Интернет-Магазин»

Студент: Комисарик Михаил Александрович

Группа: ИКБО-20-23

Работа представлена к защите 23.12.2025 [подпись] /Комисарик М.А./
(подпись и ф.и.о. студента)

Руководитель: старший преподаватель Романченко А.Е.

Работа допущена к защите 23.12.25 [подпись] /Романченко А.Е./
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: Отлично

[подпись] 23.12.25 / Болбаков А.Г., к.т.н., доцент /

[подпись] 23.12.25 / Романченко А.Е., ст.преп. /

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей,
проводивших защиту курсовой работы)

М. РТУ МИРЭА. 2025



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине: Проектирование и разработка серверных частей интернет-ресурсов
по профилю: Разработка и дизайн компьютерных игр и мультимедийных приложений
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Комисарик Михаил Александрович

Группа: ИКБО-20-23

Срок представления к защите: 12.12.2025

Руководитель: Романченко Алексей Евгеньевич, старший преподаватель

Тема: Серверная часть веб-приложения «Интернет-Магазин»

Исходные данные: используемые технологии: HTML5, CSS3, Java, JetBrains IntelliJ IDEA, SQL, наличие: межстраничной навигации, внешнего вида страниц, соответствующего современным стандартам веб-разработки, использование паттерна проектирования (MVC). Нормативный документ: инструкция по организации и проведению курсового проектирования СМК МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Провести анализ предметной области разрабатываемого веб-приложения. 2. Обосновать выбор технологий разработки веб-приложения. 3. Разработать архитектуру веб-приложения на основе выбранного паттерна проектирования. 4. Реализовать слой серверной логики веб-приложения с применением выбранной технологии. 5. Реализовать слой логики базы данных. 6. Разработать слой клиентского представления веб-приложения 7. Создать презентацию по выполненной курсовой работе.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: [подпись] /Р. Г. Болбаков/, « 15 » 09 2025 г.

Задание на КР выдал: [подпись] /А.Е. Романченко/, « 15 » сентября 2025 г.

Задание на КР получил: [подпись] /М.А. Комисарик/, « 15 » сентября 2025 г.

УДК 004

Комисарик М.А.

Курсовая работа по дисциплине «Проектирование и разработка серверных частей интернет-ресурсов» направления подготовки 09.03.04. «Программная инженерия» на тему «Серверная часть веб-приложения «Интернет-Магазин»» (5-й семестр) / руководитель ст. преп. Романченко А.Е. / кафедра ИиППО Института ИТ РТУ МИРЭА – 27 стр., 8 рис., 4 лист. 20 ист.

Целью данной курсовой работы является проектирование и разработка серверной и клиентской частей локального мультимедиа игры «Интернет-Магазин».

В результате выполнения работы произведен анализ предметной области разрабатываемого приложения, обоснован выбор технологий разработки, разработана архитектура приложения на основе выбранного паттерна проектирования, реализован слой серверной логики приложения с применением выбранной технологии, реализован слой логики базы данных, разработан слой клиентского представления приложения, создана презентация по выполненной курсовой работе.

Komisarik M.A.

Course project in the discipline “Design and development of server parts of Internet resources” of the degree program 09.03.04 “Software Engineering” on the topic “Server Side of the Web Application ‘Online Store’” (5th semester) / supervisor: Senior Lecturer Romanchenko A.E. / Department of Information and Industrial Software Engineering, Institute of Information Technologies, RTU MIREA – 27 p., 8 fig., 4 listings, 20 sources.

The purpose of this course project is to design and develop the server side of the “Online Store” application.

As a result of the work, an analysis of the subject area of the developed application was performed, the choice of development technologies was justified, the architecture of the application was designed based on the selected design pattern, the server-side logic of the application was implemented using the chosen technology, the database logic layer was implemented, the client-side representation of the application was developed, and a presentation on the completed course project was prepared.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
2 ВЫБОР ТЕХНОЛОГИЙ РАЗРАБОТКИ	10
3 РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ.....	12
4 РЕАЛИЗАЦИЯ СЛОЯ СЕРВЕРНОЙ ЛОГИКИ	14
5 РЕАЛИЗАЦИЯ СЛОЯ ЛОГИКИ БАЗЫ ДАННЫХ	17
6 РАЗРАБОТКА СЛОЯ КЛИЕНТСКОГО ПРЕДСТАВЛЕНИЯ	21
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

В условиях современной цифровой экономики электронная коммерция является одним из ключевых драйверов развития бизнеса. Переход торговых процессов в онлайн-пространство позволяет предприятиям существенно расширить аудиторию, снизить операционные издержки и повысить качество сервиса. Ядром любой системы электронной торговли выступает веб-приложение, обеспечивающее взаимодействие пользователя с каталогом товаров, формирование заказов и проведение транзакций.

Разработка серверной части таких систем является важной задачей, так как именно она отвечает за надежность хранения данных, безопасность финансовых операций и корректность выполнения бизнес-логики. Применение современных архитектурных паттернов, таких как MVC, и промышленных стандартов разработки на языке Java позволяет создавать масштабируемые и отказоустойчивые решения.

В качестве предметной области реализации интернет-магазина в данной работе выбрана сфера доставки еды (пиццерия). Данный выбор обусловлен тем, что процессы заказа продукции немедленного потребления включают в себя все стандартные механизмы E-commerce (каталог, корзина, авторизация), но при этом предъявляют повышенные требования к быстродействию системы и интерактивности, что позволяет наиболее полно раскрыть потенциал разрабатываемой серверной архитектуры.

Объектом исследования являются процессы автоматизации приема и обработки заказов в системах электронной коммерции.

Предметом исследования выступают методы и программные средства разработки серверной части веб-приложений на основе архитектурного паттерна MVC и фреймворка Spring Boot.

Целью курсовой работы является проектирование и разработка серверной части веб-приложения «Интернет-Магазин» (на примере пиццерии) для автоматизации взаимодействия с клиентами.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и рассмотреть существующие аналоги систем доставки еды, выявив их достоинства и недостатки;
- обосновать выбор стека технологий для реализации серверной части;
- разработать архитектуру веб-приложения на основе паттерна проектирования MVC (Model-View-Controller);
- спроектировать и реализовать слой логики базы данных;
- реализовать слой серверной логики, включая систему аутентификации, управление корзиной и обработку заказов, используя технологию Java;
- разработать слой клиентского представления веб-приложения с использованием серверной шаблонизации, технологий HTML5, CSS3 и JavaScript;
- обеспечить возможность развертывания системы в изолированной среде Docker.

В работе использовались методы системного анализа, объектно-ориентированного программирования и моделирования данных.

Практическая значимость работы заключается в создании полнофункционального прототипа информационной системы, готового к развертыванию и эксплуатации. Разработанное приложение позволяет пользователям регистрироваться, формировать заказы и отслеживать их статус. Реализованный подход с использованием Docker-контейнеризации упрощает процесс тестирования системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Для анализа предметной области были изучены сайты, аналогичные по тематике курсовой работе. В качестве аналогичных ресурсов были рассмотрены: Dodo Pizza [1] (рисунок 1), Domino's Pizza [2] (рисунок 2), Яндекс Еда [3] (рисунок 3).

Dodo Pizza – сайт одной из крупнейших сетей пиццерий. Ресурс очень технологичен, обладает удобным конструктором пиццы и системой отслеживания заказа в реальном времени. Однако архитектура такой системы крайне сложна и избыточна для малого бизнеса.

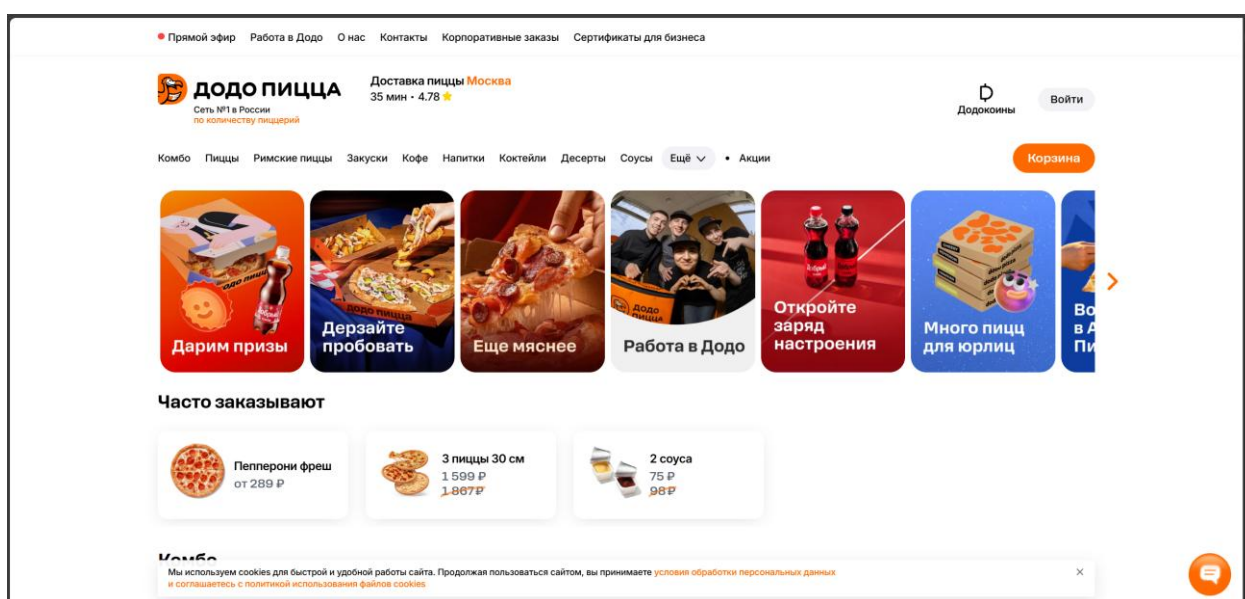


Рисунок 1 – Сайт Dodo Pizza

Domino's Pizza – сайт международной сети доставки. Он ориентирован на скорость заказа и имеет функционал «Pizza Tracker». При этом интерфейс требует обязательного указания геолокации перед просмотром меню, что снижает удобство для простого ознакомления с ассортиментом.

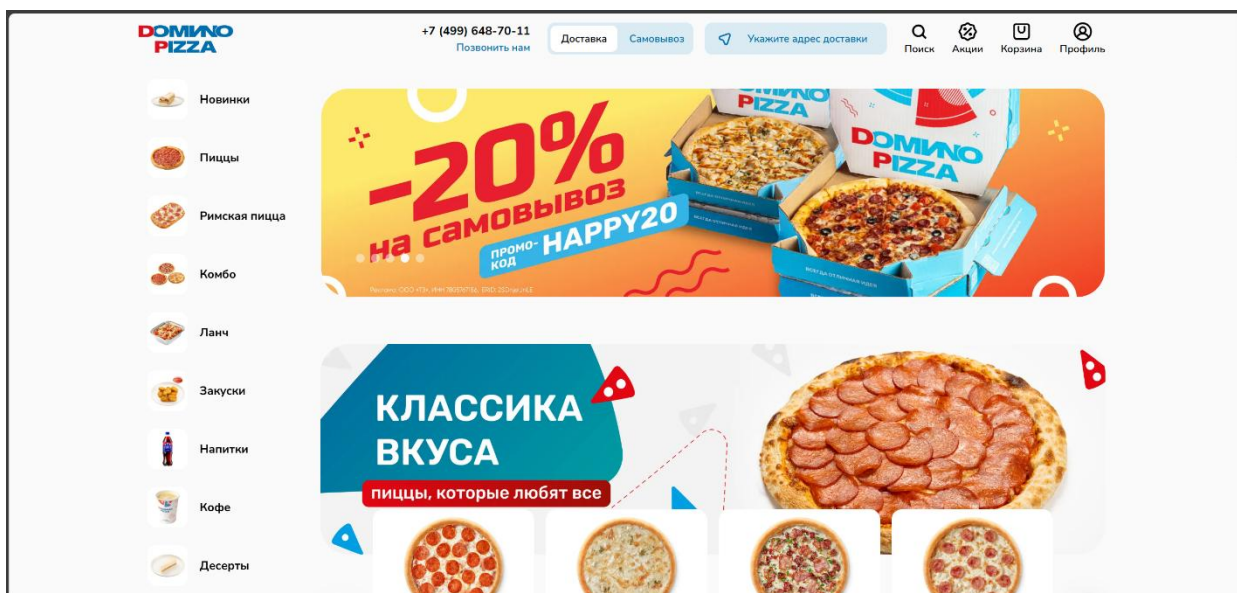


Рисунок 2 – Сайт Domino's Pizza

Яндекс Еда – популярный агрегатор доставки еды. Сайт предоставляет единый интерфейс для множества ресторанов. Несмотря на удобство для пользователя, для владельца пиццерии использование агрегатора означает отсутствие возможности развивать собственный бренд и невозможность гибкой настройки статусов заказов под свою кухню.

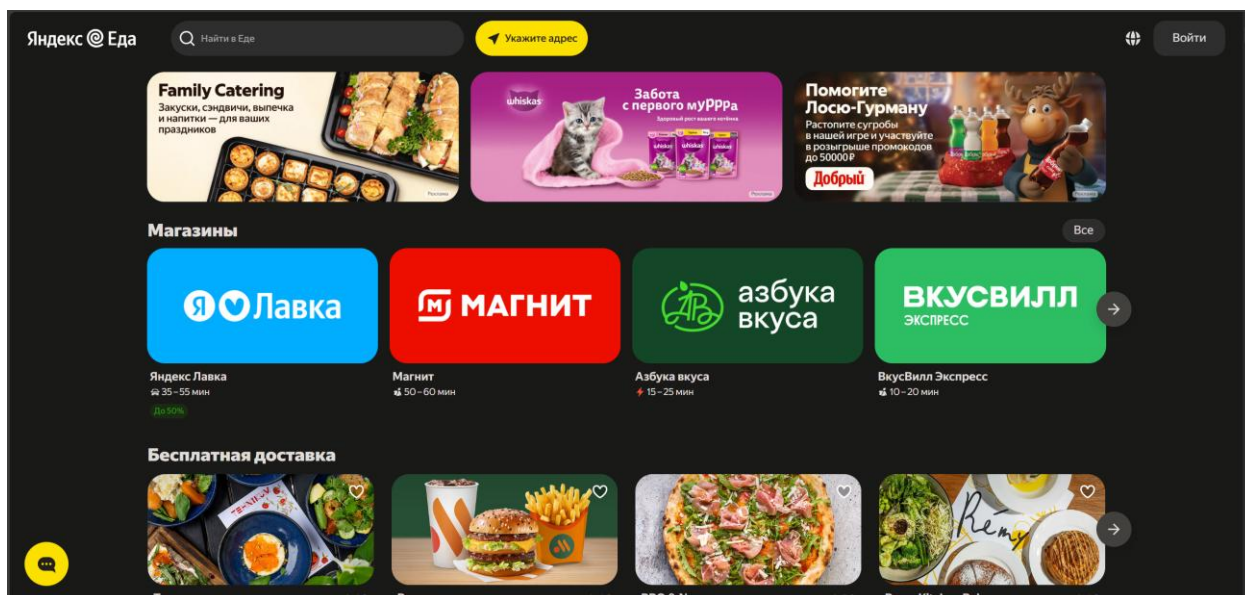


Рисунок 3 – Сайт Яндекс Еда

В ходе анализа предметной области разрабатываемого интернет-ресурса была произведена оценка целевой аудитории сайта – это клиенты пиццерии, желающие быстро оформить заказ онлайн, а также менеджеры,

обрабатывающие эти заказы. Благодаря этому выполнены ключевые запросы пользователей:

- наличие быстрой и простой навигации по меню;
- возможность добавлять товары из меню в корзину;
- возможность оформлять заказ из содержимого корзины;
- возможность отслеживать статус заказа (Новый, Готовится, Доставлен);
- наличие личного кабинета с историей заказов;
- возможность администрирования заказов менеджером;
- современный и приятный дизайн.

2 ВЫБОР ТЕХНОЛОГИЙ РАЗРАБОТКИ

Любое современное веб-приложение состоит из клиентской части, с которой взаимодействует пользователь, и серверной части, обрабатывающей данные. Для создания серверных приложений существует множество технологий, таких как PHP, Python (Django, Flask), Node.js, C# (.NET) и Java.

Для создания современных веб-приложений разработчики часто используют различные фреймворки, предлагающие набор готовых инструментов и функций. Одним из главных преимуществ использования интерпретируемых языков (Python, PHP) является высокая скорость разработки прототипов. Однако существуют и недостатки. Динамическая типизация может привести к ошибкам, которые обнаруживаются только во время выполнения программы. При масштабировании проекта такие решения могут уступать в производительности и строгости архитектуры компилируемым языкам, таким как Java.

Использование готовых CMS (систем управления контентом) позволяет быстро развернуть магазин, но ограничивает возможности кастомизации бизнес-логики, например, специфической обработки статусов заказов или интеграции с внешними системами.

Создание серверной части с использованием технологий Java и фреймворка Spring Boot является классическим и наиболее распространенным способом для разработки надежных корпоративных систем и веб-приложений, поэтому для разработки сайта был выбран этот набор технологий. Также, согласно заданию, необходимо реализовать паттерн MVC, для чего идеально подходит связка Spring Boot и шаблонизатора Thymeleaf.

Java — строго типизированный объектно-ориентированный язык программирования. Приложения на Java транслируются в специальный байт-код, поэтому они могут работать на любой компьютерной архитектуре с помощью виртуальной машины Java. Это обеспечивает высокую надежность и переносимость [4].

Spring Boot — это фреймворк на основе Java, который упрощает создание готовых к работе приложений. Он берет на себя управление зависимостями и автоматическую конфигурацию, позволяя разработчику сосредоточиться на бизнес-логике, а не на настройке окружения. Spring Boot реализует архитектуру MVC и предоставляет встроенные средства безопасности [5].

PostgreSQL — мощная объектно-реляционная система управления базами данных с открытым исходным кодом. Она выбрана за надежность, соответствие стандартам SQL и способность обрабатывать сложные запросы, необходимые для работы интернет-магазина [6].

Docker — программное обеспечение для автоматизации развертывания и управления приложениями в средах с поддержкой контейнеризации. Использование Docker позволяет упаковать приложение и базу данных в изолированные контейнеры, гарантируя, что оно будет работать одинаково на любой машине [7].

Для развертывания и запуска веб-приложения используется технология Docker Compose, позволяющая управлять многоконтейнерным приложением с помощью одного конфигурационного файла.

В качестве интегрированной среды разработки (IDE) была выбрана JetBrains IntelliJ IDEA, так как она является стандартом для разработки на Java, поддерживает все необходимые фреймворки (Spring, Hibernate) и форматы файлов, а также имеет удобные инструменты для работы с базами данных и Docker. Лицензия на версию Ultimate предоставляется бесплатно для студентов.

3 РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ

Архитектура разрабатываемой системы представляет из себя трехуровневую клиент-серверную архитектуру (рисунок 4). В качестве клиента выступает веб-браузер пользователя, отображающий интерфейс и отправляющий запросы. Сервер приложения обрабатывает бизнес-логику и генерирует HTML-страницы. Сервер баз данных обеспечивает надежное хранение и целостность данных.

Организация системы выполнена с использованием инструмента контейнеризации Docker. Приложение и база данных функционируют в независимых контейнерах, объединенных в единую виртуальную сеть посредством Docker Compose. Это обеспечивает изоляцию компонентов и упрощает развертывание на любой платформе.

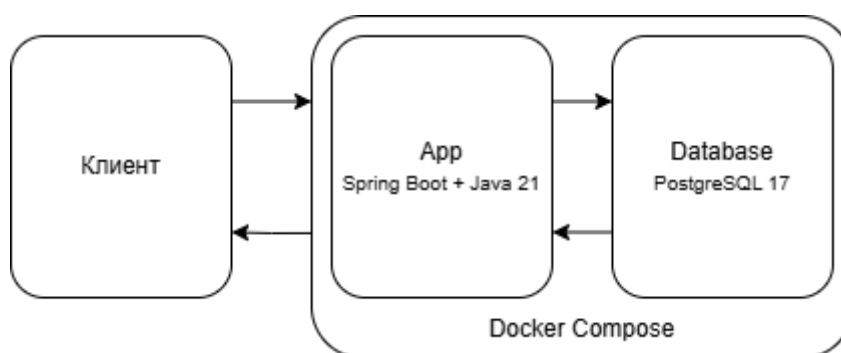


Рисунок 4 – Трехуровневая архитектура

Архитектура серверной части построена на базе фреймворка Spring Boot и реализует паттерн проектирования MVC (Model-View-Controller) [8] (рисунок 5). Данный подход подразумевает четкое разделение ответственности между компонентами системы:

Уровень представления (View/Controller). Обрабатывает входящие HTTP-запросы от клиента. Контроллеры (AuthController, OrderController) принимают данные, проводят первичную валидацию и передают управление бизнес-логике. Для генерации ответа используется шаблонный движок Thymeleaf, который формирует HTML-страницы на стороне сервера.

Уровень бизнес-логики (Service Layer). Инкапсулирует основные алгоритмы работы системы (OrderService, UserService). Здесь происходят транзакционные операции, расчет стоимости заказов и управление статусами. Сервисы не зависят от деталей реализации веб-интерфейса или базы данных.

Уровень доступа к данным (Data Access Layer). Реализован с использованием Spring Data JPA. Репозитории (Repository) обеспечивают абстракцию над базой данных PostgreSQL, преобразуя объекты Java (Entity) в SQL-запросы и обратно.

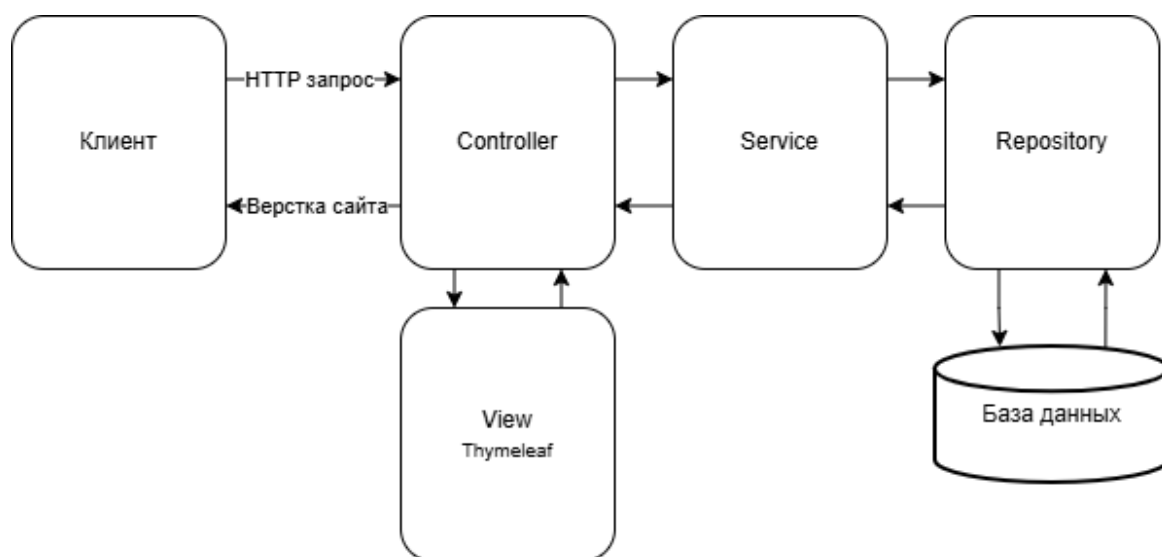


Рисунок 5 – Архитектура MVC внутри Spring

В архитектуре клиентского представления, используется гибридный подход. Основная структура страниц формируется на сервере, однако для обеспечения интерактивности (работа корзины, динамическое обновление счетчиков) используется слой клиентской логики на JavaScript.

Для исключения дублирования кода в шаблонах используется механизм фрагментов. Общие элементы интерфейса (шапка, навигация) вынесены в отдельные модули, которые подключаются ко всем страницам, что упрощает поддержку и масштабирование системы.

4 РЕАЛИЗАЦИЯ СЛОЯ СЕРВЕРНОЙ ЛОГИКИ

Серверная логика приложения реализована в сервисном слое, компоненты которого внедряются в контроллеры посредством механизма инъекции зависимостей [5]. Данный архитектурный подход обеспечивает слабую связность модулей и упрощает модульное тестирование. Бизнес-логика системы структурно разделена на подсистемы безопасности, управления пользователями и обработки заказов.

Подсистема безопасности построена на принципах Stateless-архитектуры с использованием стандарта JWT [9]. В отличие от сессионной модели, состояние авторизации не сохраняется в памяти сервера. При успешной аутентификации генерируется токен, содержащий идентификационные данные пользователя и криптографическую подпись. Интеграция механизма аутентификации в контекст Spring Security выполнена путем внедрения пользовательского фильтра JwtAuthFilter в цепочку фильтров Spring Security [10] перед стандартным фильтром UsernamePasswordAuthenticationFilter. Это обеспечивает проверку токена до момента доступа к контроллерам, позволяя сформировать объект SecurityContext с правами текущего пользователя. Обработка попыток несанкционированного доступа делегирована компоненту RedirectAuthenticationEntryPoint, который обеспечивает перенаправление пользователя на страницу входа с сохранением целевого URL.

Ключевой бизнес-процесс оформления заказа реализован в классе OrderService. Взаимодействие с контроллером осуществляется через объекты передачи данных (DTO), что позволяет исключить передачу избыточных полей сущностей. Для обеспечения целостности входных данных используется механизм валидации на основе спецификации Bean Validation. Поля объектов DTO аннотированы ограничениями @NotBlank, @Size и @Email, что позволяет предотвратить обработку заведомо некорректных данных на раннем этапе выполнения запроса. В случае нарушения

ограничений генерируется исключение, которое перехватывается глобальным обработчиком ошибок. Метод создания заказа выполняется в едином транзакционном контексте, обеспечиваемом аннотацией `@Transactional` [11]. Это гарантирует атомарность операции: сохранение заказа и связанных с ним позиций происходит либо в полном объеме, либо не происходит вовсе, что предотвращает нарушение целостности данных.

В процессе обработки заказа производится верификация цен товаров путем запроса к базе данных, расчет итоговой стоимости и формирование записей в таблице позиций заказа. Алгоритм создания заказа представлен в листинге 1.

Листинг 1 – Метод создания заказа в классе `OrderService`

```
@Transactional
public void createOrder(User user, OrderRequestDto
request)
{
    Order order = new Order();
    order.setUser(user);
    order.setAddress(request.getAddress());
    order.setPhoneNumber(request.getPhoneNumber() !=
null ? request.getPhoneNumber() :
user.getPhoneNumber());
    order.setComment(request.getComment());
    order.setStatus(OrderStatus.NEW);

    double total = 0;

    for (OrderRequestDto.OrderItemDto itemDto :
request.getItems())
    {
        Product product =
productRepository.findById(itemDto.getProductId())
.orElseThrow(() -> new RuntimeException("Product not
found"));

        OrderItem item = new OrderItem(order,
product, itemDto.getQuantity(), product.getPrice());
```

Продолжение Листинга 1

```
        order.getItems().add(item);

        total += product.getPrice() *
itemDto.getQuantity();
    }

    order.setTotalPrice(total);
    orderRepository.save(order);
}
```

Управление жизненным циклом заказа осуществляется администратором через соответствующий контроллер. Для типизации состояний заказа используется перечисление `OrderStatus`, что исключает ошибки ввода данных и упрощает локализацию интерфейса. Изменения статуса, производимые администратором, синхронизируются с базой данных и отображаются в пользовательском интерфейсе.

Разработанный слой серверной логики обеспечивает корректную обработку бизнес-транзакций, валидацию входных данных и разграничение прав доступа пользователей.

5 РЕАЛИЗАЦИЯ СЛОЯ ЛОГИКИ БАЗЫ ДАННЫХ

Проектирование схемы базы данных выполнено с использованием подхода Code-First, при котором структура реляционных таблиц генерируется автоматически на основе объектной модели приложения. Для реализации объектно-реляционного отображения использована библиотека Hibernate [12], реализующая спецификацию JPA. Конфигурация подключения к СУБД PostgreSQL и параметры генерации схемы (DDL) определены в файле настроек приложения.

Инфологическая модель системы включает четыре основные сущности. Сущность User предназначена для хранения учетных данных и контактной информации пользователей. В качестве первичного ключа используется уникальный идентификатор UUID, поля электронной почты и номера телефона индексируются как уникальные. Сущность Product описывает товарные позиции меню, включая название, описание, стоимость и относительный путь к медиа-контенту. Конфигурация полей сущностей включает явное определение ограничений на уровне базы данных посредством аннотаций `@Column(nullable = false)` и `@Column(unique = true)`, что гарантирует консистентность данных даже при прямом доступе к СУБД.

Центральным элементом схемы является сущность Order, агрегирующая информацию о статусе выполнения, адресе доставки и итоговой стоимости заказа. Связь с пользователем реализована через отношение «многие-к-одному». Детализация состава заказа осуществляется через сущность OrderItem, связанную с Order отношением «один-ко-многим». Для обеспечения целостности данных при операциях сохранения и удаления настроено каскадирование типа ALL. Сущность OrderItem фиксирует стоимость товара на момент оформления транзакции, что обеспечивает корректность исторических данных при изменении цен в каталоге [13].

Реализация класса сущности Order представлена в листинге 2.

Листинг 2 – Класс сущности Order

```
@Data
@Entity
@Table(name = "orders")
@NoArgsConstructor
public class Order
{
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false)
    private String address;

    @Column(nullable = false)
    private String phoneNumber;

    @Column
    private String comment;

    @Enumerated(EnumType.STRING)
    private OrderStatus status = OrderStatus.NEW;

    private Double totalPrice;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "order", cascade =
CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new
ArrayList<>();
}
```

Визуальное представление схемы базы данных и связей между сущностями приведено на рисунке 6.

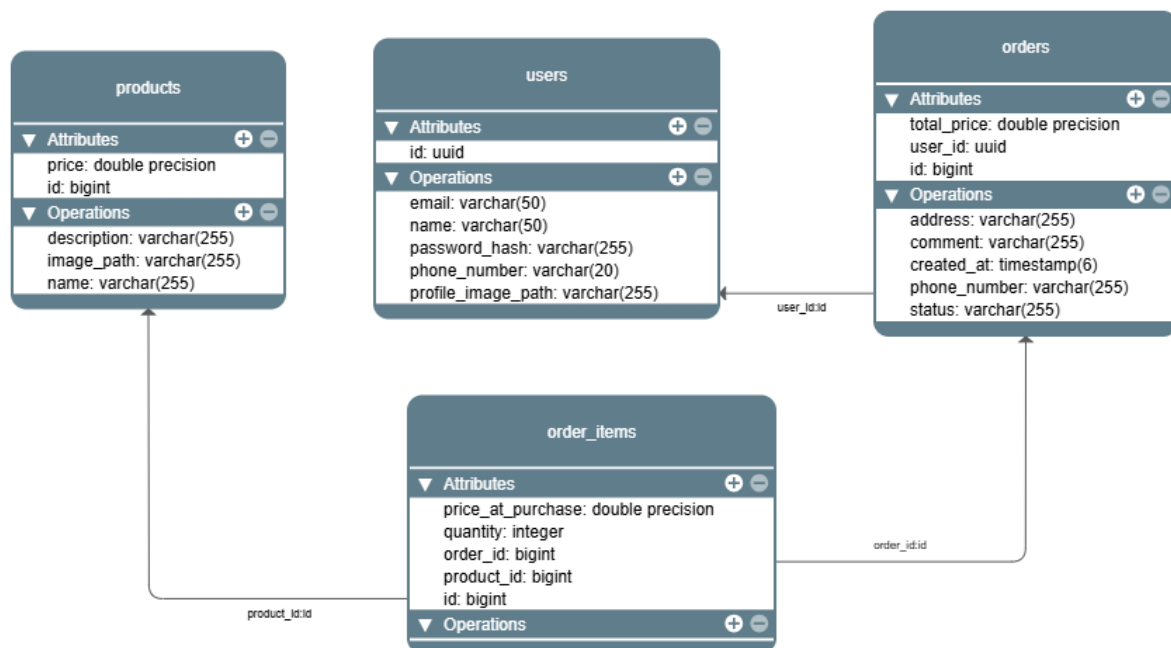


Рисунок 6 – ER диаграмма базы данных

Взаимодействие с базой данных осуществляется через слой репозитория. Разработанные интерфейсы наследуют функциональность JpaRepository [14], предоставляя методы для выполнения CRUD-операций. Для реализации выборок, выходящих за рамки стандартных CRUD-операций, использован механизм деривации запросов Spring Data JPA. Метод `findAllByUserOrderByCreatedAtDesc` автоматически транслируется в SQL-запрос с условием `WHERE user_id = ?` и сортировкой `ORDER BY created_at DESC`, что избавляет от необходимости написания явных запросов на языке JPQL.

Для первичного наполнения базы данных реализован компонент `DatabaseLoader`. При инициализации контекста приложения данный компонент проверяет состояние таблицы товаров и, в случае отсутствия записей, выполняет импорт предустановленных данных, сопоставляя их с файлами изображений, доступными в подключенном томе файловой системы. Это обеспечивает готовность системы к эксплуатации непосредственно после развертывания.

Фрагмент реализации компонента загрузки данных представлен в листинге 3.

Листинг 3 – Инициализация данных в DatabaseLoader

```
@Override
public void run(String... args) throws Exception {
    if (productRepository.count() > 0) return;

    List<PizzaInitData> pizzas = List.of(
        new PizzaInitData("Пепперони", "Классическая
пицца...", 549, "pepperoni.png"),
        new PizzaInitData("Маргарита", "Томаты и
моцарелла...", 449, "margarita.png")
    );

    for (PizzaInitData data : pizzas) {
        savePizza(data);
    }
}
```


6 РАЗРАБОТКА СЛОЯ КЛИЕНТСКОГО ПРЕДСТАВЛЕНИЯ

Разработка клиентской части приложения выполнена с использованием стандартного стека веб-технологий HTML5, CSS3 и JavaScript. Для генерации динамического контента на стороне сервера применяется шаблонный движок Thymeleaf [15], обеспечивающий интеграцию данных Java-объектов в HTML-разметку. Адаптивность интерфейса и кроссбраузерная совместимость достигаются за счет использования CSS-фреймворка Bootstrap 5 [16].

В целях оптимизации кодовой базы и соблюдения принципа DRY (Don't Repeat Yourself) реализован модульный подход к верстке. Общие элементы интерфейса, такие как навигационная панель, заголовок и подвал страницы, вынесены в отдельный файл фрагментов fragments.html. Данные компоненты динамически подключаются ко всем страницам веб-приложения с использованием атрибута th:replace, что упрощает поддержку и внесение изменений в дизайн системы. Стилизация элементов вынесена в отдельный файл каскадных таблиц стилей style.css.

Реализация корзины товаров выполнена по гибридной схеме. До момента оформления заказа состояние корзины сохраняется в локальном хранилище браузера [17], что позволяет снизить нагрузку на сервер и обеспечить мгновенный отклик интерфейса. Интерактивные элементы, такие как счетчики количества товаров, обновление индикатора корзины и расчет промежуточной стоимости, управляются скриптами на языке JavaScript [18]. Скрипты осуществляют манипуляцию DOM-деревом (Document Object Model) в реальном времени. При изменении состава корзины происходит пересчет итоговой суммы и обновление визуального индикатора количества товаров в навигационной панели без полной перезагрузки страницы. Взаимодействие с сервером при оформлении заказа осуществляется асинхронно посредством Fetch API [19], отправляющего структурированный JSON-объект с данными заказа на REST-контроллер. Клиентская логика включает обработку HTTP-кодов ответов сервера: в случае получения кода 401 (Unauthorized) или 403

(Forbidden) пользователь автоматически перенаправляется на страницу авторизации с сохранением параметров редиректа.

Пользовательский интерфейс включает несколько ключевых экранных форм. Главная страница представляет собой каталог продукции с возможностью добавления товаров в корзину. Визуальное представление карточки товара с элементами управления количеством приведено на рисунке 7.

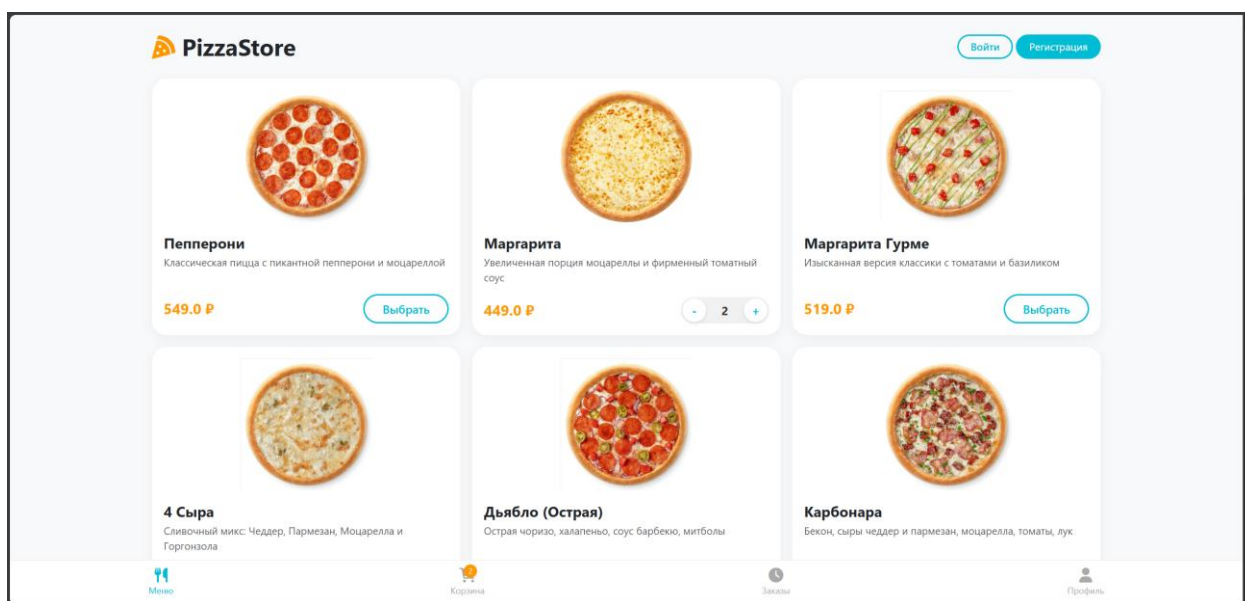


Рисунок 7 – Интерфейс страницы меню

Страница оформления заказа реализует функционал автозаполнения контактных данных для авторизованных пользователей, получая информацию из серверной модели. Личный кабинет предоставляет доступ к истории заказов и настройкам профиля. Внешний вид страницы оформления заказа представлен на рисунке 8.

Рисунок 8 – Интерфейс страницы оформления заказа

Фрагмент HTML-кода страницы оформления заказа, демонстрирующий интеграцию серверных данных в поля формы, приведен в листинге 4.

Листинг 4 – Фрагмент шаблона страницы оформления заказа

```
<div class="card p-3 border-0 shadow-sm rounded-4 mb-4">
  <div class="mb-3">
    <label class="small text-muted fw-bold">Имя</label>
    <input type="text" class="form-control"
id="nameInput"
      th:value="${user != null ? user.name : ''}" required>
  </div>

  <div class="mb-3">
    <label class="small text-muted fw-bold">Номер
телефона</label>
    <input type="tel" class="form-control"
id="phoneInput"
      th:value="${user != null ?
user.phoneNumber : ''}"
      placeholder="+7 (999) 000-00-00"
required>
  </div>

  <div class="mb-3">
```

Продолжение Листинга 4

```
        <label class="small text-muted fw-  
bold">Email</label>  
        <input type="email" class="form-control"  
id="emailInput"  
th:value="${user != null ? user.email  
: ''}">  
    </div>  
</div>
```

Для администратора системы разработан интерфейс управления заказами. Табличное представление данных позволяет отслеживать поступающие заявки и изменять их статус. Визуальная индикация статусов реализована через динамическое присвоение CSS-классов в зависимости от состояния объекта заказа.

Разработанный слой клиентского представления обеспечивает корректный доступ ко всему функционалу серверной части.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы проведен анализ предметной области, оценка существующих веб-ресурсов для заказа доставки еды. Выявлены необходимые пользователям функциональные возможности и роли в системе. Определены цели и задачи для разработки серверной части интернет-ресурса. Выбраны современные технологии разработки.

На этапе проектирования разработана архитектура веб-приложения на основе паттерна MVC, учитывающая потребности в масштабируемости и безопасности. Спроектирована схема базы данных, обеспечивающая целостность хранения информации о пользователях и заказах.

Реализованы слои серверной логики и доступа к данным при помощи технологий Java, Spring Boot и Hibernate. Разработан слой клиентского представления с использованием шаблонизатора Thymeleaf. Реализована система аутентификации и авторизации пользователей. Обеспечена возможность развертывания системы в изолированной среде с помощью технологии Docker.

Исходный код разработанного веб-приложения размещен в открытом репозитории системы контроля версий GitHub [20].

Таким образом, была разработана серверная часть веб-приложения «Интернет-Магазин», отвечающая основным потребностям пользователей и администраторов.

Отчет по курсовой работе выполнен в соответствии с ГОСТом 7.32-2017 [21].

Все поставленные цели и задачи выполнены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Сеть пиццерий Dodo Pizza [Электронный ресурс]. – URL: <https://dodopizza.ru> (дата обращения: 26.11.2025).
2. Domino's Pizza – доставка пиццы [Электронный ресурс]. – URL: <https://dominospizza.ru> (дата обращения: 26.11.2025).
3. Яндекс Еда – сервис заказа еды [Электронный ресурс]. – URL: <https://eda.yandex.ru> (дата обращения: 26.11.2025).
4. Шилдт, Г. Java. Полное руководство : пер. с англ. / Г. Шилдт. – 12-е изд. – Москва : Диалектика, 2022. – 1344 с.
5. Уоллс, К. Spring в действии : пер. с англ. / К. Уоллс. – 6-е изд. – Москва : ДМК Пресс, 2022. – 598 с.
6. PostgreSQL Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/> (дата обращения: 26.11.2025).
7. Моуэт, Э. Использование Docker. Разработка и внедрение программного обеспечения : пер. с англ. / Э. Моуэт. – Москва : ДМК Пресс, 2017. – 354 с.
8. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма [и др.]. – Санкт-Петербург : Питер, 2020. – 368 с.
9. RFC 7519 - JSON Web Token (JWT) [Электронный ресурс]. – URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 26.11.2025).
10. Spring Security Reference [Электронный ресурс]. – URL: <https://docs.spring.io/spring-security/reference/index.html> (дата обращения: 26.11.2025).
11. Фаулер, М. Шаблоны корпоративных приложений : пер. с англ. / М. Фаулер. – Москва : Вильямс, 2020. – 544 с.
12. Hibernate ORM 6.4 User Guide [Электронный ресурс]. – URL: https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/Hibernate_User_Guide.html (дата обращения: 26.11.2025).

13. Кузнецов, С. Д. Основы баз данных : учебное пособие / С. Д. Кузнецов. – 2-е изд. – Москва : БИНОМ, 2007. – 484 с.
14. Spring Data JPA - Reference Documentation [Электронный ресурс]. – URL: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/> (дата обращения: 26.11.2025).
15. Thymeleaf Documentation [Электронный ресурс]. – URL: <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html> (дата обращения: 26.11.2025).
16. Bootstrap 5.3 Documentation [Электронный ресурс]. – URL: <https://getbootstrap.com/docs/5.3/getting-started/introduction/> (дата обращения: 26.11.2025).
17. Web Storage API - MDN Web Docs [Электронный ресурс]. – URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API (дата обращения: 26.11.2025).
18. Флэнаган, Д. JavaScript. Подробное руководство : пер. с англ. / Д. Флэнаган. – 7-е изд. – Санкт-Петербург : Символ-Плюс, 2021. – 704 с.
19. Fetch API - MDN Web Docs [Электронный ресурс]. – URL: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (дата обращения: 26.11.2025).
20. Репозиторий исходного кода веб-приложения «Интернет-Магазин» [Электронный ресурс]. – URL: https://github.com/krezerenko/pizzeria_website (дата обращения: 16.12.2025).
21. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Введ. 2018-07-01. – Москва : Стандартинформ, 2017. – 28 с.